MCaM : Efficient LLM Inference with Multi-tier KV Cache Management

Kexin Chu , Zixu Shen , Sheng-Ru Cheng , Dawei Xiang , Ziqin Liu , and Wei Zhang*

School of Computing, University of Connecticut
{kexin.chu, qzt24001, fvi24001, ieb24002, ziqin.liu, wei.13.zhang}@uconn.edu

***Abstract*—The KV cache in current LLM serving system is primarily used to accelerate processing within a single request and is aggressively deleted once the response is generated. However, in scenarios like virtual assistants and multi-turn conversations, the KV cache can be reused across requests, which can dramatically reduce computation costs and improve serving latency. Caching historical tokens, however, significantly increases memory requirements. Furthermore, existing serving systems treat the request scheduler and KV cache separately, despite their tight coupling.**

MCaM is a multi-tier cache system that enables the KV cache reuse and sharing across requests. It leverages DRAM as slow-tier memory for storing the KV cache of historical prompts. To efficiently utilize fast-tier Hign Bandwidth Memory(HBM) on GPU, we co-designed the KV cache manager and scheduler to coordinate request scheduling and token placement across tiers. To hide the reload time, MCaM employs a pipeline prefetcher that overlaps communication and computation. Additionally, MCaM incorporates a quality-aware sparsification algorithm to heterogeneously compress the KV cache in each layer. This approach not only reduces data transfer size but also decreases the overall KV cache size. To remove data offloading from a request's critical path, we designed an asynchronous offload engine that swaps data from HBM to DRAM in the background. Our experiments show that MCaM can reduce TTFT by up to 69% and improve prompt prefilling throughput by 3.3X. It can also reduce the end-to-end latency of LLM inference by up to 58% when request length increase to 4096 tokens.

I. Introduction

In recent years, transformer-based Large Language Models (LLMs) have been shown to have excellent performance in solving many complex NLP tasks [7], [15], [41]. These successes has led to a surge in LLM-based serving systems, such as ChatGPT, DeepSeek, and CoPilot. These LLM serving systems are deployed at increasingly larger scales and are expected to provide low latency and high throughput, spurring greater demand for optimizing inference performance [3], [42], [33], [30], [16], [37].

LLM inference typically operates in two stages [43], [52], [2], [18], [19]: *prefill* stage, where the attention states of the input sequence are computed and stored in the *Key-Value cache (KV cache)*; and *decode* stage, where the system generates text auto-regressively using the stored KV cache. However, the KV cache occupies a considerable portion of GPU high-bandwidth memory (HBM), since the attention states are cached for all input tokens, so the memory size of the associated KV cache grows linearly with the length of the context sequence [40], [12]. This can severely limit the system's ability to handle longer sequences or multiple concurrent requests efficiently.

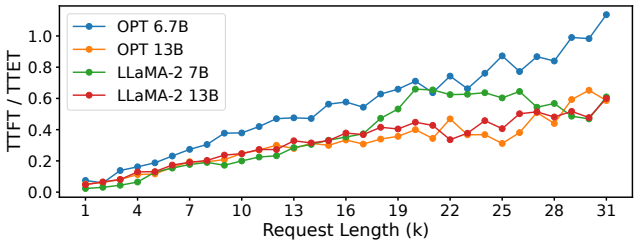
To address these challenges and limit the memory usage of the KV cache, current LLM serving systems implement a strategy where the KV cache is directly discarded once the request is completed [4], [22], [36], freeing up GPU memory but preventing reuse, leading to redundant computations and increased latency for similar requests. As illustrated in Fig. 1, reusing KV caches from previous requests plays a critical role in reducing the time to first token (TTFT) by shrinking the prefill stage, which dominates latency for long inputs. Since prefill cost scales with input length, reuse becomes increasingly valuable for long-context LLMs, improving both response time and system throughput by avoiding redundant computation [10], [47].

Prior work [13], [32] primarily targets intra-session reuse, leveraging cached KV entries within the same conversation. However, many real-world workloads—such as ChatGPT prompts or enterprise applications—feature repeated prefixes across sessions (e.g., static system prompts or task instructions). Recomputing these across requests incurs unnecessary overhead. While some methods [51], [20] enable cross-session reuse within HBM, GPU memory remains a bottleneck. Its limited capacity restricts the number of cache entries that can be retained, especially under high load or long-context scenarios. This motivates a more scalable and memory-efficient cache system that can support wide reuse across sessions without being constrained by GPU memory limits.

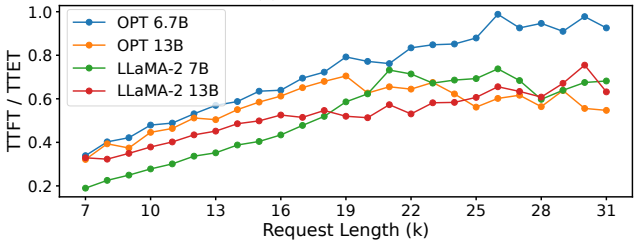
To accommodate the historical KV cache, especially for long prompts [6], [9], [11], we have to leverage external memory [24] and wisely manage the KV cache placement across the tiered memory. However, this bring several challenges: 1) where to place the KV cache across the tiers? 2) How to limit the data transfer overhead across the tiers? 3) When should we kick off the data transfers between the tiers?

To address the above challenges, we propose MCaM , a multi-tier cache system designed to wisely manage the KV cache of historical prompts, reuse them across requests, and efficiently migrate data between the fast-tier HBM and the slow-tier DRAM [39], [44], [21]. By implementing a sophisticated caching strategy, MCaM ensures that frequently accessed data

* Corresponding Author.



(a) ShareGPT



(b) LongAlign

Fig. 1: Proportion of latency between the prefilling stage (TTFT) and decoding stage(TTET). (a) is based on the dataset ShareGPT [34], (b) is based on the dataset LongAlign [5]

remains in the faster HBM, while less frequently accessed data can be stored in the slower, larger DRAM. This strategic data placement allows the system to balance speed and capacity effectively. As shown in Fig. 2, reloading the KV caches from slow-tier memory to HBM can improve prefill latency by at least 30% compared to directly recomputing these tokens, which consumes a significant amount of GPU computational resources. This improvement motivates our MCaM design to further enhance efficiency and reduce latency. By leveraging the capabilities of MCaM , we can achieve better utilization of memory resources, resulting in faster response times and improved system performance. Our contributions are:

- We identify the main sources of latency when processing long prompts and the major design defects in existing LLM serving systems.
- We build a multi-tier KV cache system, designated as MCaM , to enable KV cache reuse across requests, utilizing the larger DRAM as supplemental storage to store the historial KV caches.
- We co-design the scheduler and cache management system, encompassing techniques such as scoring-based cache replacement, pipelined prefetching, asynchronous offload, and cache-aware scheduling, to achieve high throughput for LLM inference.
- We propose an improved algorithm for compressing the KV cache in each layer. Which can reduce the KV cache size, thereby mitigating storage stress and improving data transmission from the DRAM to the GPU.

II. Background and Motivation

A. KV cache in LLM

Current LLMs use Transformer architectures with key-value (KV) caching in attention mechanisms, but KV cache size grows linearly with sequence length, increasing memory and computation costs. To improve efficiency, various methods have been proposed to optimize KV cache memory for high-throughput serving. vLLM [22] and vAttention [31] minimize KV cache fragmentation by storing KV caches contiguously with demand paging, enabling cache sharing and larger batch processing. InfiniGen [23] offloads KV caches to CPU memory and selectively prefetches key entries to the GPU.

To further reduce storage overhead, MiniCache [26] and PyramidInfer [46] use depth-dimension compression or layer-level compression to improve inference efficiency. While H2O [50], Scissorhands [28], Keyformer [1], and FastGen [14] considering token reduction method to minimize KV cache size by retaining essential tokens. Additionally, Q-Hitter [49] further optimizes this through quantization. CacheGen [27] encodes KV caches into compact bitstreams, adapting compression levels to network bandwidth constraints.

Request scheduling is also crucial for optimizing LLM serving performance. Various strategies have been proposed to enhance scheduling efficiency [29], [38], [48], [8], [35], [52]. For multi-instance scheduling, [29] employs a round-robin policy, while [38] incorporates dynamic load balancing, memory migration, and prioritization of important requests.

B. Opportunities and Challenges

In some scenarios, similar contexts appear repeatedly [**?**], such as identical requests or prefix tokens from different users in virtual assistant applications, or prior interactions required in multi-round conversation apps like ChatGPT. This repetition leads to processing numerous tokens during the prefill stage, causing heavy computational workload. As shown in Fig. 1, longer prompts increase the TTFT share of the end-to-end latency (TTET). Given the prevalence of long prompts in complex tasks, optimizing the prefill stage of LLM serving systems is crucial.

By managing historical tokens and sharing KV caches across requests, we can significantly reduce tokens processed during prefill. However, storing these KV caches requires substantial memory. For instance, a batch of 32 requests with 2K tokens can occupy approximately 83.2GB of KV cache using OPT-30B, compared to 56GB for model parameters. Due to limited and costly GPU high-bandwidth memory (HBM), we must offload unused KV caches to DRAM, enabling reuse but incurring data migration. Reusing KV caches from DRAM reduces TTFT by at least 30%(Fig. 2), demonstrating the effectiveness of cache reuse.

Managing data placement and migration between HBM and DRAM is challenging due to speed and cost differences. Key challenges include: 1.determining data placement across tiers given varying KV cache sizes and migration costs; 2.limiting migration overhead to prevent GPU stalls or HBM overfill; 3. determining the timing of prefetch to avoid GPU resource occupation caused by premature fetch or GPU waiting caused by untimely fetch. Therefore, co-designing the cache manager and scheduler is essential to optimize data placement, mini-

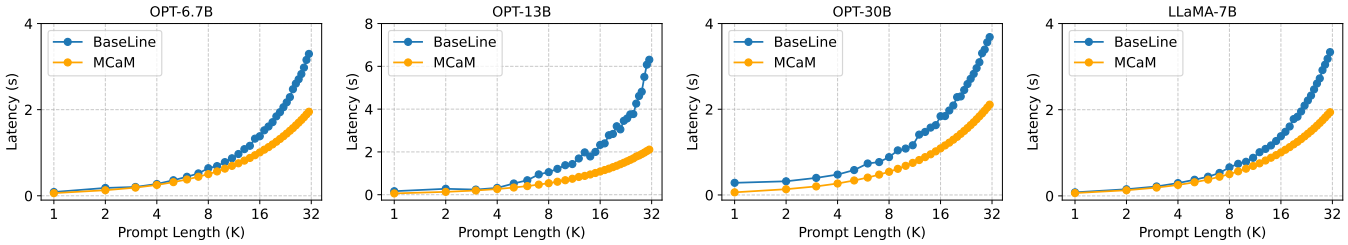


Fig. 2: The latency comparison between recompute and load from DRAM.

mize migration overhead, and ensure data is loaded into HBM precisely when needed.

III. System Design

A. Overview

In this section, we introduce MCaM , a multi-tier cache system designed to support efficient reuse of historical KV caches across requests. MCaM maintains a structured cache table that records whether a prompt's KV cache resides in GPU's HBM or host DRAM. When a cache hit occurs, the corresponding KV cache is directly retrieved, bypassing the compute-intensive prefill stage. This significantly reduces prefill latency and GPU computation, improving overall GPU throughput and LLM inference efficiency. The system architecture is shown in Fig. 3.

Unlike traditional systems that decouple memory management and scheduling, MCaM adopts a **co-designed architecture** that tightly integrates both components. A score-based cache policy dynamically manages data movement between HBM and DRAM, ensuring that frequently accessed caches remain in fast memory for optimal performance. To further reduce latency, MCaM features a **pipelined loading engine** that loads KV caches layer-by-layer, overlapping data transmission with GPU execution. In parallel, an **asynchronous offload engine** moves stale caches to CPU memory in the background, avoiding interference with active tasks. Additionally, MCaM incorporates a **quality-aware compression module** that sparsifies KV caches based on attention scores. This reduces transfer overhead and memory usage while preserving key information needed for accurate inference.

Together, these mechanisms allow MCaM to significantly enhance LLM inference efficiency by reducing redundant computation, optimizing memory usage, and improving responsiveness across varying request patterns.

B. Scheduler & Cache Manager Co-design

Scheduler. As illustrated in Fig. 4, MCaM adopts a priority-based scheduling mechanism to manage request dispatching once the GPU becomes available. The scheduler maintains a priority queue, and only the requests within a bounded **shuffle window** are considered eligible for execution. These requests are sorted by priority to maximize cache reuse and minimize GPU idle time. A critical design decision is how to prioritize these requests. MCaM defines three tiers of priority:

- **Highest**: Requests that can reuse KV cache in GPU to minimize prefill latency and avoid unnecessary memory transfers.
- **Medium**: Requests without reusable KV cache, since they do not require prefetching or DRAM-to-HBM transfer. It can help prevent the GPU from remaining idle (Waiting for prefetching).
- **Lowest**: Requests requiring historical KV cache from DRAM, which may delay execution due to DRAM-to-HBM transfers.

Within the high and medium priority groups, requests follow a **first-come, first-served (FCFS)** policy to maintain fairness. In the low priority group, requests are further ranked by the size of the KV cache they need to load—the larger the cache, the lower the scheduling priority. This mitigates the risk of GPU underutilization caused by large, slow-to-transfer requests.

To prevent starvation of low-priority requests, MCaM introduces a maximum waiting time threshold. If a request's wait time exceeds this limit, it is promoted to the shuffle window, a fixed-size staging buffer where requests are held until dispatched. Inside the shuffle window, priorities are no longer re-evaluated, preventing frequent reordering that could disrupt cache prefetching and increase cache miss rates.

The shuffle window size is calibrated based on the GPU's maximum supported prompt length, ensuring that prefetched KV cache blocks can be efficiently buffered without exceeding HBM constraints. This co-design of cache management and scheduling enables MCaM to achieve high throughput while maintaining low latency and fairness across diverse request patterns.

Cache Management. To tightly integrate cache management with the scheduling subsystem, MCaM maintains a centralized metadata table that tracks key attributes of all active KV cache entries, as illustrated in Fig. 4. Each matadata record includes the following fields:

- **NodeID**: A globally unique identifier (e.g., a hash) assigned to each KV cache node.
- **Addr**: The physical address where the KV cache is stored, either in CPU DRAM or GPU HBM.
- **Size**: The memory size occupied by this KV cache block.
- **Valid_in_GPU**: A binary flag indicating whether the KV cache is currently resident in GPU memory.

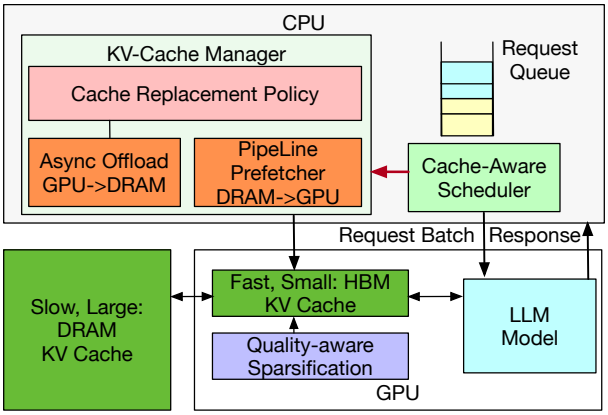


Fig. 3: The System Architecture of MCaM

- **Score**: A dynamically updated importance score used to guide eviction and replacement decisions.

All KV cache blocks are organized into fixed-length chunks and hierarchically using a Trie-Tree structure, enabling fast prefix matching and block-level reuse. When a new request is issued, MCaM traverses the Trie from the root node to locate matching prefix nodes. For each matched node, it checks the metadata table to verify if the corresponding KV cache is already resident in GPU memory.

If some required cache blocks reside only in CPU memory, the system collects their physical addresses and initiates asynchronous prefetching to transfer them into GPU memory. During this process, the system dynamically manages HBM allocation and updates the metadata table accordingly. Once the transfer is complete, the $Valid\_in\_GPU$ flag is set to true for those blocks, making them available for computation.

This mechanism ensures that only the necessary KV cache blocks are prefetched, reducing transmission latency and conserving HBM space. The Score field plays a crucial role under memory pressure, helping MCaM make informed decisions about which blocks to retain or evict based on usage frequency and expected reuse. By integrating fine-grained metadata tracking, Trie-guided prefix matching, asynchronous transfer, and score-aware eviction, MCaM enables high KV cache hit rates and low latency. This coordination between cache and scheduler components ensures the LLM inference pipeline remains efficient, even under complex multi-user workloads with dynamic request patterns.

Prefetcher. When processing a batch of concurrent requests, the GPU must load all relevant and reusable KV cache nodes into its high-bandwidth memory (HBM) to accelerate the prefill stage. However, due to the asynchronous nature of request execution and varying data availability, some KV cache nodes may arrive later than others. In such cases, the GPU may be forced to idle while waiting for pending data transfers, introducing unnecessary latency and making KV cache loading a bottleneck, even in the presence of prefill optimizations.

To address this challenge, MCaM integrates a predictive prefetching strategy that monitors future KV cache requirements and proactively initiates data movement from DRAM to GPU memory. Specifically, for each request within the **Shuffled Window**, the scheduler inspects its associated $NodeID$ and consults the centralized cache metadata table. If the node's $Valid\_in\_GPU$ flag is already set to $True$, it means the KV cache block is available in HBM, and no further action is needed. Otherwise, if the block resides in DRAM, the scheduler extracts its $Addr$ and $Size$ fields and sends this information to the prefetching engine for immediate processing.

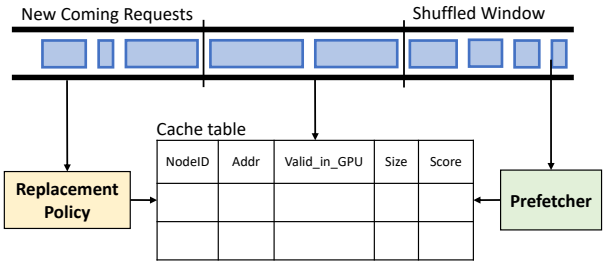


Fig. 4: The Co-design of Scheduler and Cache Manager

To optimize prefetch timing, the system predicts request completion using the LLM's estimated output length and pre-collected $TPOP$. A timer is set when a request reaches the GPU, updating after each decode iteration by subtracting $TPOP$. Using this, MCaM predicts completion times and of estimates the KV cache loading latency for the next request. Since LLMs process layer by layer, we can obtain the required transmission latency by:

$$trans = \frac{SIZE}{bandwidth * num\_layers}$$

Where $SIZE$ is the size of the required KV cache nodes obtained from the cache table, $bandwidth$ is the bandwidth of PCIe, and num_layers indicates the number of layer of the used LLMs. Periodically, the prefetcher compares the loading latency $trans$ with the nearest completion time of requests on GPU and obtain the proper loading start time.

$$fetch(t) = \mathbb{I}_{\min_{0 \le i \le N}\{TPOP \times len_i - (t - timer_i)\} \le trans}(t)$$

Here, N represents the total number of requests currently being handled by the GPU, t denotes the current time, len_i is the predicted output length for request i.

Cache Replacement Policy. We propose a score-based cache replacement policy to evict KV cache nodes from HBM when it reaches full capacity, aiming to minimize unnecessary data transfers. The policy selects the node with the smallest $Score$ as the eviction candidate. When a request enters the **Shuffled Window**, the scheduler checks whether its KV cache node is already in GPU memory by varifying the $Valid\_in\_GPU$ flag. If the node is present, its $Score$ is increased by 1, reflecting its immediate reuse. After the request finishes its autoregressive generation phase, the node's $Score$ is decreased by 2 to counterbalance the earlier increments. If multiple nodes share the lowest score, one is randomly selected for

eviction. This policy ensures that frequently reused KV cache nodes remain in GPU memory, improving cache efficiency and reducing the overhead of repeated transfers from DRAM.

C. Efficient Data Transmission Module

Layer-wise Pipeline Loading. To prevent KV cache node loading from becoming a performance bottleneck, we employ prefetching as the primary strategy for mitigating latency. However, when the GPU is idle and the prefetcher has not yet issued a load request, the GPU should be able to directly read data from DRAM. To enable this, we design a flexible loading module that can be accessed by the prefetcher and the GPU. Which enables tight coordination between data transfer and computation, allowing memory I/O and GPU execution to proceed in parallel and improving overall data readiness.

Given that LLM inference progresses in a layer-wise manner, preloading the entire KV cache into HBM prior to execution introduces unnecessary memory pressure and latency. To address this, MCaM , adopts a layered pipeline approach, in which KV cache loading is interleaved with model execution. Specifically, the KV cache is divided into per-layer blocks, and each block is copied from DRAM to HBM as an independent transfer subtask. These subtasks are enqueued and executed sequentially by the loading module, enabling the GPU to begin prefill computation for the current layer while subsequent layers' KV cache is being asynchronously loaded.

One key challenge in overlapping execution and data transfer is minimizing GPU idle time, especially under CUDA's limitation of non-preemptible asynchronous copy operations. Long-running transfer tasks may delay time-critical requests if not handled carefully. To alleviate this, MCaM further fragments each layer-wise KV cache transfer into smaller, fine-grained subtasks. This fine-grained structure allows the scheduler to insert high-priority transfers at the front of the queue, which are executed immediately after the current subtask completes. This opportunistic preemption reduces contention between concurrent tasks and ensures that urgent requests can bypass stalled transfers, thereby improving overall system responsiveness.

Asynchronous Data Offloading. In vLLM's existing implementation, when a request lack sufficient enough HBM space for its KV cache, it is blocked while the KV caches is offloaded to host memory, causing excessive I/O operations and system performance degradation. To optimize this process and eliminate the offloading overhead on the request's critical path, we introduce an asynchronous data offloading mechanism, which proactively offload KV cache nodes before HBM reaches its limit, following a predefined replacement policy. This process runs in the background, ensuring offloading does not block other operations. Once offloaded to DRAM, the cache is released from HBM, allowing the GPU and load module to operate without interruption, improving overall efficiency. Additionally, when an offloading task is initiated, a success signal is sent to MCaM , enabling the GPU to continue inference or prefetch required KV cache data. This approach ensures better resource management, reduces latency under high loads, and enhances overall performance and user experience.

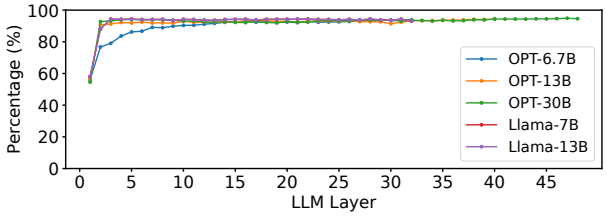


Fig. 5: The sparsity rate for each layer in the LLM models.

D. Quality-aware Compression

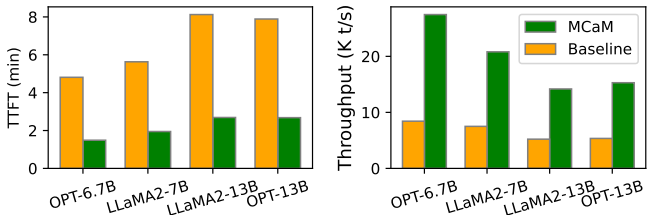
Since the KV cache size grows linearly with the number of tokens, reloading long contexts during inference introduces significant data transmission overhead. Most LLMs are pretrained on input sequences with limited length, and when deployed on inputs exceeding this maximum, they often exhibit degraded performance. This "out-of-domain" generalization issue arises because the model has never been exposed to such long contexts during pretraining, leading to inefficiencies and even incorrect predictions or degraded output quality in downstream tasks.

To address this challenge, data compression presents a practical and effective solution. As illustrated in Fig. 5, the attention score matrix computed via the equation softmax $\left(\frac{QK^T}{\sqrt{d_k}}\right)$ demonstrates high sparsity, consistent with prior observations [50], [45], [25], [17], [27]. When elements below 1% of a row's maximum value are considered negligible, the first layer already shows approximately 60% sparsity, which increases to around 95% in deeper transformer layers. This sparsity indicates that most attention scores contribute minimally to inference, suggesting that compressing low-importance key-value pairs could substantially reduce computation and transmission without harming model performance.

Building on this insight, we propose a context- and attention-aware KV cache compression strategy. Unlike conventional techniques like fixed-range sliding window attention, which preserve tokens solely based on positional heuristics, our method dynamically retains only the most impactful key-value vectors as determined by their attention relevance. This design enables more aggressive compression while maintaining the fidelity of inference results.

When a new LLM is initialized within the MCaM runtime, the system first conducts dry-run inference using a set of predefined prompts to analyze the sparsity patterns across each transformer layer. Based on these patterns, MCaM dynamically adapts the compression ratio during subsequent inference sessions. Specifically, it filters out key-value vectors with low attention scores and eliminates those with minimal impact on downstream computation, thereby reducing memory footprint and bandwidth consumption.

However, this design introduces new challenges in cache reuse. Prefix tokens concatenated with different suffixes can



(a) Latency (b) Throughput

Fig. 6: The Comparison of TTFT and Prefill Throughput between MCaM and Baseline. (a) TTFT represents the LLM prefill latency, and (b) The unit of prefill throughput (K t/s) mean K tokens per second.

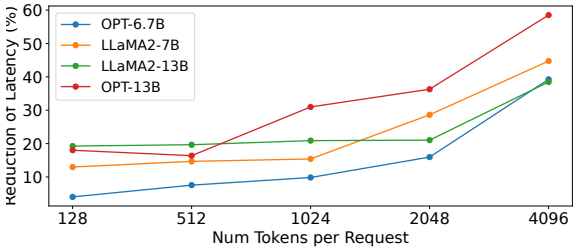


Fig. 7: The comparison of end-to-end latency between MCaM and the Baseline under different request lengths. The coordinates of y-axis is %, which indicate the extent of latency reduction that MCaM has achieved compared to the Baseline at each respective request length.

result in diverse attention score distributions, thereby altering the subset of required key-value vectors. To handle this, MCaM adopts a block-wise KV cache management scheme, wherein KV caches are partitioned and stored in fixed-size blocks. During inference, MCaM retrieves only the necessary blocks from GPU or CPU memory, depending on access frequency, offloading less relevant blocks to CPU DRAM and freeing up limited GPU HBM capacity.

By dynamically adjusting the compression level based on runtime context length and model complexity, MCaM significantly reduces transmission and storage overhead while preserving inference accuracy. This quality-aware compression mechanism enhances scalability, reduces latency, and makes the system more adaptable to diverse NLP workloads with varying prompt structures and lengths.

IV. Evaluation

In this section, we conduct a comprehensive evaluations of MCaM , analyzing its effectiveness in reducing end-to-end latency and improving throughput in LLM inference. Our experiments explore the impact of request lengths variations, different data arrival patterns, and diverse hardware environments on MCaM 's performance. Specifically, we seek to answer the following key questions:

1) How does MCaM perform across different prompt lengths and models compared to the baseline (§ IV-B)?
2) How do different data arrival distribution affect MCaM 's latency and throughput (§ IV-B)?
3) How does MCaM perform under various hardware configurations and shared node sizes (§ IV-C)?
4) How does co-designing the cache manager and scheduler influence the cache hit rate and performance (§ IV-C)?
5) What is the impact of MCaM 's quality-aware compression on inference accuracy and computational overhead (§ IV-D)?

A. Experiment Setup

Our experiments were conducted on systems equipped with NVIDIA A100 GPUs (40GB), 1TB of DRAM, and PCIe Gen 4 interconnects. MCaM was implemented in Python and CUDA using vLLM [22], with block-based memory management to optimize storage utilization and reduce fragmentation. To ensure efficient data transfers between the GPU and DRAM, we employed dedicated CUDA streams and separate threads, enabling continuous batching for improved performance.

For evaluation, we used workloads derived from the ShareGPT dataset [34] and LongAlign [5], consisting of 5,000 mixed-length dialogue sessions with token lengths ranging from 128 to 4096 (±15%). We tested MCaM on the 6.7B and 13B OPT models, as well as the 7B and 13B LLaMA-2 models, all configured with FP16 weights and MCaM intermediate activations. To benchmark its performance, we compared MCaM against a baseline approach implemented in vLLM.

B. Overall Performance

In this section, we present the performance improvements brought by MCaM . The primary objective of MCaM is to minimize prefill latency (TTFT) in LLM inference by leveraging KV cache reuse, thereby improving the end-to-end performance of LLM inference. Our experiments first focus on the performance improvement in the prefill phase. Second, as shown in Fig. 2, KV-cache reuse can achieve better performance with longer request lengths. Therefore, the next set of experiments investigates the performance of MCaM by varying data arrival distributions. Finally, we examine MCaM 's performance with mixed-length requests, which better reflect real-world usage scenarios.

Effect of Prefill Stage Optimization. Here, we evaluate the prefill latency and throughput of MCaM across different model configurations. In our experiments, we selected 2.5k requests of varying lengths, using the first 1k requests as a warm-up phase to evaluate TTFT across different models.

TTFT. As shown in Fig. 6 (a), compared to the baseline, MCaM significantly reduces TTFT for OPT-6.7B, LLaMA2-7B, LLaMA2-13B, and OPT-13B by 69%, 65.4%, 66.9%, and 66.1%, respectively. This reduction is primarily attributed to MCaM 's ability to eliminate redundant computations in generating the KV cache for repeated tokens during the prefill phase via KV cache reuse. Consequently, upon a cache hit, the TTFT of MCaM is determined solely by the time required to compute

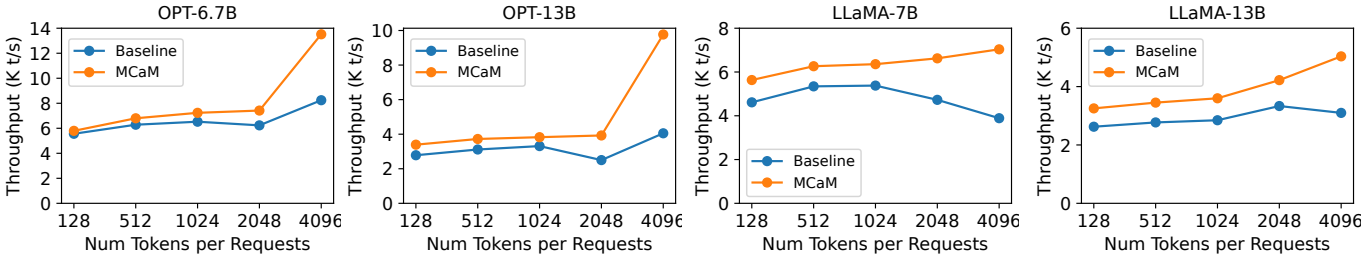


Fig. 8: The comparison of Throughput between MCaM and Baseline, the unit is K tokens/s

the KV cache for the unshared suffix tokens in the requests. Moreover, as the model size increases, the performance gains offered by MCaM becomes more pronounced.

Prefill throughput. Prefill throughput is another critical metric for assessing how efficiently an LLM serving system processes incoming prompts at scale. As illustrated in Fig. 6 (b), MCaM delivers substantial improvements over the baseline approach, achieving throughput speedups of 3.3x, 2.8x, 2.7x, and 2.9x for OPT-6.7B, LLaMA2-7B, LLaMA2-13B, and OPT-13B, respectively.

Impact of Request Length Variability. As shown in Fig. 1, effective cache reuse can significantly reduce redundant computations, particularly for longer prompts. To empirically validate this observation, we selected five distinct sets of requests with varying lengths from ShareGPT and LongAlign. Within each set, the first 1k requests were designated as a warm-up phase to prime the system, followed by systematic measurement of performance on the remaining requests.

The end-to-end latency, defined as the total time from a user submitting a prompt to the LLM delivering a complete response, is shown in Fig. 7. Compared to the baseline approach, MCaM achieves substantial improvements in latency across all prompt length categories. For example, in the case of OPT-13B, MCaM reduces the end-to-end latency by 58.5%. Moreover, as the average request length increase, the latency reduction becomes increasingly pronounced, growing from under 20% to over 38%. This upward trend aligns closely with our preliminary observations, and further reinforces the benefit of KV cache reuse for longer prompts. As modern LLMs continue evolving toward supporting substantially longer context windows, the relative advantage provided by MCaM is expected to increase accordingly.

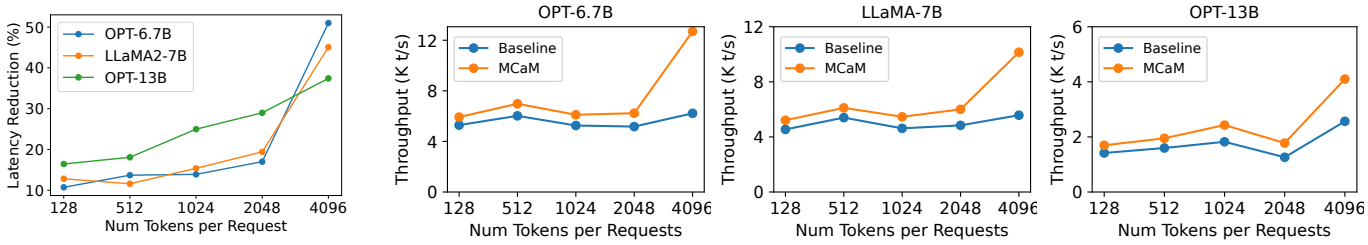
The comparison of end-to-end throughput is presented in Fig. 8. Our results indicate that MCaM consistently improves token processing throughput relative to the baseline as prompt lengths grow. Notably, when the prompt length approaches 4096 tokens, the throughput improvement ranges from 1.6× to 2.4× depending on the model. This performance gain in throughput is primarily attributed to two factors: (1) reduced TTFT by eliminating redundant KV cache computations for repeated tokens, and (2) an increased effective batch size—up to 1.8× for LLaMA2-7B—enabled by higher cache reuse and shared context across requests. Collectively, these factors contribute to significantly enhanced system efficiency under longer-context workloads.

Impact of Data Arrival Distribution. While MCaM effectively reduces redundant computation in the prefill phase by reusing KV caches from shared prefix tokens, the actual performance is also influenced by the distribution pattern of request arrivals. In the worst-case scenario, incoming requests may consist of multiple disjoint groups, each characterized by distinct prefix tokens. This situation forces the system to frequently load different KV cache entries into GPU HBM, incurring additional communication overhead that may diminish the performance benefits of cache reuse. To quantify this effect, we conducted an additional experiment using a uniformly distributed arrival pattern to simulate less favorable request locality.

The comparison of end-to-end latency under this setup is illustrated in Fig. 9 (a). Although the improvements in latency for specific prompt lengths are less pronounced than those observed in Fig. 7, where requests follow their natural order from the ShareGPT dataset—the overall performance trend remains consistent with our expectations. For instance, when the prompt length is $\leq$ 2048, the latency reduction for models such as OPT-13B and LLaMA2-7B is relatively modest, ranging from 1.6% to 9.2%. This reduction is smaller than in the original order-preserved scenario. However, despite the less favorable arrival distribution, MCaM still provides tangible performance gains. As prompt lengths increase, the advantages of KV cache reuse become increasingly evident, with latency reductions reaching up to 50% for OPT-6.7B, 45% for LLaMA2-7B, and 37% for LLaMA2-13B.

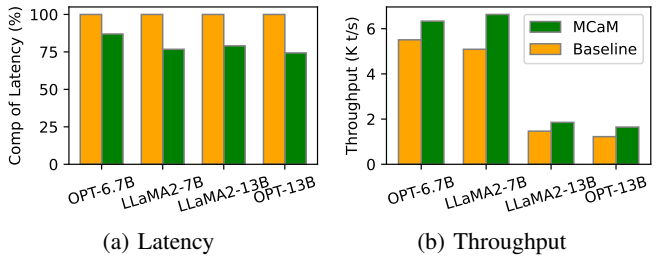
The corresponding comparison of throughput is shown in Fig. 9 (b). Similar to the latency analysis, we observe that throughput improvements grow consistently with request length. Even under uniformly distributed arrivals, the system achieves an overall throughput enhancement ranging from 1.6× to 2.0×. These results confirm that MCaM maintains its effectiveness across diverse traffic patterns and remains a robust optimization for long-context LLM inference.

Impact of Mixed-Length Prompts. In the previous subsections, we evaluated MCaM 's performance using request batches of fixed lengths. However, in practical deployment scenarios, LLM serving systems must simultaneously handle a diverse mix of request lengths, as user interactions vary widely across sessions. To evaluate such real-world workload characteristics, we selected 5K dialogue sessions from ShareGPT



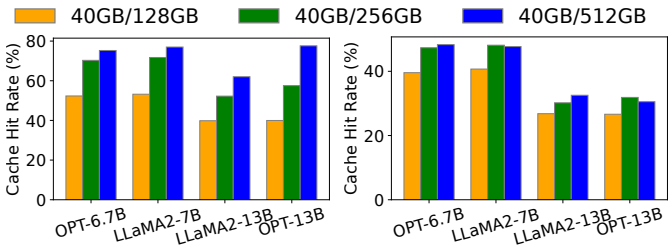
(a) Latency (b) Throughput

Fig. 9: The comparison of latency and throughput between MCaM and Baseline, with the request arrival distribution is uniform.



(a) Latency (b) Throughput

Fig. 10: The comparison of latency and throughput between MCaM and Baseline under mixed-length prompts



(a) Overall (b) HBM

Fig. 11: The cache hit rate of MCaM under different CPU memory size. (a) records the overall cache hit rate of the MCaM , and (b) records the cache hit rate on the GPU HBM.

and expanded them into multiple rounds of back-and-forth conversations. The first 20% of requests were designated as a warm-up phase to allow system stabilization and cache priming prior to performance evaluation. This setup aims to reflect the complexity and heterogeneity present in large-scale online inference workloads.

As depicted in Fig. 10, MCaM achieves noticeable improvements in both latency and throughput when processing mixed-length inputs. Specifically, the end-to-end latency was reduced by 13%, 23.3%, 21%, and 25.7% for OPT-6.7B, LLaMA2-7B, LLaMA2-13B, and OPT-13B, respectively. Correspondingly, the throughput increased by 1.15x, 1.3x, 1.27x, and 1.33x across these models. These gains highlight the benefit of partial KV cache reuse, even under irregular and unpredictable prompt lengths.

Nonetheless, the observed performance improvements were somewhat lower than our initial expectations. This outcome can be attributed to the distribution of prompt lengths within the ShareGPT dataset. Based on dataset statistics, more than 75% of the prompts consist of 512 tokens or fewer; only 6.1% exceed 1,024 tokens, and a mere 0.6% are longer than 2048 tokens. In prior experiments, we have shown that MCaM yields higher benefits for longer prompts, as they offer greater opportunities for cache reuse and computational savings. When restricted to short inputs—especially those below 512 tokens—MCaM 's latency reduction tends to be modest, often below 20% across models and below 8% for OPT-6.7B.

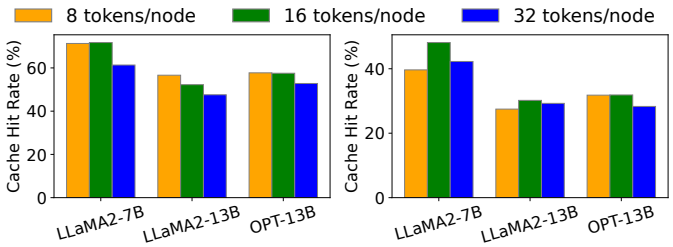
Despite these limitations, the results remain encouraging. As LLMs evolve to support substantially longer context windows and as user inputs become more complex, the effectiveness of MCaM is expected to increase. This underscores the scalability and future relevance of cache-aware memory optimization in LLM inference systems.

C. Performance of Co-design

In § IV-B, we evaluated MCaM 's performance improvements over the baseline under varing prompt lengths and data arrival distributions. However, hardware configurations—such as CPU memory capacity—can also significantly affect the performance of MCaM . In this section, we analyze these effects through a series of targeted experiments. Since MCaM accelerates LLM inference by storing and reusing the KV cache of prior requests, the cache hit rate becomes a critical metric for evaluating overall system effectiveness. We present results on how cache hit rate varies with hardware settings to highlight its impact on end-to-end performance.

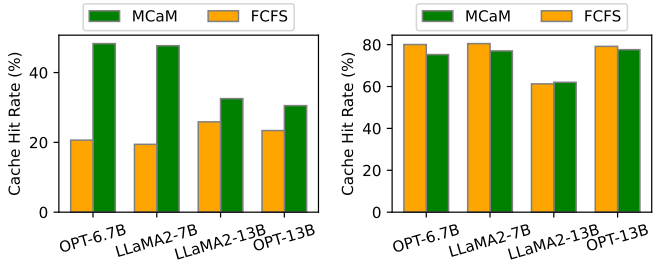
Impact of CPU Memory Size. MCaM employs a multi-tiered memory hierarchy, where both GPU high-bandwidth memory (HBM) and host-side DRAM are utilized to store KV caches. A larger CPU memory allows for a more extensive historical cache to be maintained, increasing the opportunities for reuse and improving overall inference efficiency. To assess this impact, we conducted experiments measuring the cache hit rate of MCaM across different host memory configurations.

As shown in Fig. 11 (a), the overall cache hit rate increases with larger CPU memory sizes. This result aligns with expectations: with more available host DRAM, a greater portion of historical KV caches can be retained and prefetched into



(a) Overall (b) HBM

Fig. 12: The cache hit rate for different node sizes. The node here represents the smallest token block that can be shared.(a) records the cache hit rate on GPU HBM, and (b) records the overall cache hit rate of MCaM



(a) HBM's Cache Hit Rate (b) Overall Cache Hit Rate

Fig. 13: The cache hit rate of FCFS and our cache-aware scheduling strategy.

GPU memory, boosting reuse rates. Notably, we also observe a positive correlation between CPU memory and the GPU HBM cache hit rate, which reaches up to 48% for LLaMA2-7B and 32% for OPT-6.7B, driven primarily by MCaM 's proactive prefetching strategy.

However, the cache hit rates for larger models such as LLaMA2-13B and OPT-13B remain comparatively lower. This discrepancy is due to their significantly higher KV cache footprint, which limits the number of tokens that can be cached within a fixed memory budget. Specifically, LLaMA2-13B and OPT-13B require approximately 1.56× more storage per token compared to their smaller counterparts, LLaMA2-7B and OPT-6.7B. As a result, under identical CPU memory settings, fewer tokens are retained, leading to reduced cache reuse and lower overall hit rates.

Impact of Minimum Reusable Node Size. As discussed earlier, MCaM employs a trie tree structure for cache management, where each node stores fixed-length blocks of tokens. This setup enables varying degrees of token reuse, with smaller node sizes offering finer granularity and potentially greater token reuse for a single request. However, in practical scenarios, LLM inference systems handle multiple concurrent requests. Over-optimizing for a single request can lead to contention for GPU storage space, especially for the prefix tokens of other requests, resulting in increased cache misses and frequent data transfers.

To determine the optimal node size, we conducted a series of experiments to compare cache hit rates across different node sizes. As illustrated in Fig. 12, the overall cache hit rate of MCaM decreases rapidly as the node size increases. This outcome aligns with expectations, as larger node sizes result in fewer tokens being reusable across multiple requests. For instance, with a node size of 32, a prompt length of 17 cannot be reused, but with a node size of 16 or smaller, this portion of the KV cache can be effectively reused.

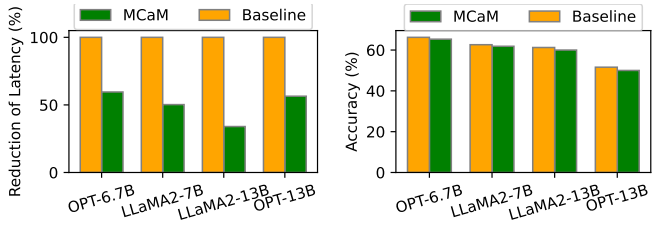
Additionally, when examining the cache hit rate on HBM, we observed that a node size of 16 yielded the best results compared to the other configurations. Consequently, all other experiments in this study adhered to this node size setting.

Impact of Scheduling Strategy: FCFS vs. Cache-Aware Scheduling. Our cache management and scheduler co-design introduces a cache-aware scheduling strategy that explicitly aims to maximize KV cache reuse by prioritizing request ordering. This design ensures that KV Cache entries residing in GPU HBM can be reused to the greatest possible extent, thereby reducing the frequency of costly data transfers between the GPU and the host system. To access the effectiveness of this strategy, we conducted a series of controlled experiments comparing the conventional First-Come-First-Serve (FCFS) scheduling algorithm, as adopted in the baseline, with our proposed cache-aware scheduling approach.

As illustrated in Fig. 13, the cache-aware scheduling strategy leads to a substantial increase in the GPU HBM cache hit rate compared to the FCFS baseline. Specifically, for the OPT-6.7B, LLaMA2-7B, LLaMA2-13B, and OPT-13B models, the cache hit rates improve of 27.7%, 28.3%, 6.7%, and 7.2%, respectively. The relatively smaller gains observed for LLaMA2-13B and OPT-13B can be attributed to their hifher per-token memory footprint, which restricts the number of tokens that can be effectively shared within the same available HBM capacity.

Furthermore, when examining the aggregate cache hit rate across all requests, we find that the FCFS strategy performs nearly on par with our cache-aware counterpart. This observation suggests that our strategy can significantly enhance token reuse and memory efficiency within the GPU HBM without sacrificing overall cache hit performance. In addition, it alleviates redundant data movement between the GPU and host memory, there by improving overall system throughput and resource utilization.

Impact of Asynchronous Data Transfer. To mitigate the blocking effects of data transfer between the GPU and host during LLM inference tasks, MCaM utilizes separate CUDA streams and dedicated data transfer threads for asynchronous data transmission. We performed a set of experiments to compare our asynchronous data transfer scheme with the traditional data transfer method used in the vLLM baseline. As shown in Fig. 14 (a), our approach significantly reduces data transfer delays by 50% - 66%, leading to a substantial performance improvement in LLM inference.



(a) Data transfer latency (b) Accuracy

Fig. 14: (a).Data transfer latency between async & baseline; (b).Accuracy between compression & baseline.

D. Compression Overhead

As demonstrated in Fig. 5, the attention score matrices in each layer of LLM inference exhibit significant sparsity. This sparsity serves as a key motivation for implementing KV cache compression in MCaM . By selectively retaining important tokens based on their attention significance, MCaM effectively reduces the number of tokens that need to be stored and processed. However, this token reduction may raise concerns about potential impacts on the accuracy of LLM inference. In this section, we conduct a series of experiments to assess both the effect of KV cache compression on inference accuracy and the computational overhead it introduces.

Accuracy. The influence of KV cache compression on accuracy is illustrated in Fig. 14 (b). When applying a 50% overall compression ratio,MCaM incurs a maximum accuracy degradation of only 0.9% across models. This minor impact is likely attributed to the inherent sparsity patterns of LLMs, where the initial layers tend to exhibit lower sparsity levels and are thus less affected by token reduction. For tasks requiring higher accuracy, users can adopt a more conservative compression ratio to further minimize the potential loss. These findings highlight the importance of tailoring compression levels to the characteristics of target tasks, enabling an effective balance between inference efficiency and prediction quality.

Overhead The compression process in MCaM involves computing importance scores from the attention matrices to identify tokens that contribute most significantly to downstream predictions. Although this step introduces some additional computation, the latency overhead remains modest—accounting for just 1.36% of the total inference latency. This is negligible when compared to the substantial end-to-end performance improvements of 13% to 25.7% observed in previous experiments. As such, our design demonstrates that incorporating lightweight compression mechanisms into cache management can yield considerable performance benefits with minimal overhead.

V. Conclusion

This paper presents MCaM , a multi-tier cache management system designed to enable effective reuse and sharing of the KV cache across inference requests. MCaM substantially reduces the recomputation overhead associated with KV cache generation in LLMs. To maximize reuse efficiency, MCaM incorporates hierarchical KV cache placement, overlapped cache transmission, a cache-aware scheduling strategy, and a quality-aware compression algorithm. Extensive experiments show that MCaM reduces the time-to-first-token(TTFT) by up to 69% and improves prompt prefill throughput by up to 3.3x. It also lowers the end-to-end inference latency by as much as 58% for requests with lengths up to 4096 tokens.

References

[1] Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant Nair, Ilya Soloveychik, and Purushotham Kamath. Keyformer: Kv cache reduction through key tokens selection for efficient generative inference. *Proceedings of Machine Learning and Systems*, 6:114–127, 2024.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, 2024.

[3] Sohaib Ahmad, Hui Guan, Brian D Friedman, Thomas Williams, Ramesh K Sitaraman, and Thomas Woo. Proteus: A high-throughput inference-serving system with accuracy scaling. 2024.

[4] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, SC '22. IEEE Press, 2022.

[5] Yushi Bai, Xin Lv, Jiajie Zhang, Yuze He, Ji Qi, Lei Hou, Jie Tang, Yuxiao Dong, and Juanzi Li. Longalign: A recipe for long context alignment of large language models. *arXiv preprint arXiv:2401.18058*, 2024.

[6] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.

[8] Ke Cheng, Wen Hu, Zhi Wang, Hongen Peng, Jianguo Li, and Sheng Zhang. Slice-level scheduling for high throughput and load balanced llm serving. *arXiv preprint arXiv:2406.13511*, 2024.

[9] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[10] Kexin Chu, Tzechinh Liu, Yunding Li, Pengchao Yuan, and Wei Zhang. Car: An efficient kv cache reuse system for large language model inference. *Workshop llm-gnn.org*, 2024.

[11] Jiayu Ding, Shuming Ma, Li Dong, Xingxing Zhang, Shaohan Huang, Wenhui Wang, Nanning Zheng, and Furu Wei. Longnet: Scaling transformers to 1,000,000,000 tokens. *arXiv preprint arXiv:2307.02486*, 2023.

[12] Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S Kevin Zhou. Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference. *arXiv preprint arXiv:2407.11550*, 2024.

[13] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. {Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 111–126, 2024.

[14] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801*, 2023.

[15] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[16] Cong Guo, Jiaming Tang, Weiming Hu, Jingwen Leng, Chen Zhang, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–15, 2023.

[17] Zhuomin He, Yizhen Yao, Pengfei Zuo, Bin Gao, Qinya Li, Zhenzhe Zheng, and Fan Wu. Adaskip: Adaptive sublayer skipping for accelerating long-context llm inference. *arXiv preprint arXiv:2501.02336*, 2025.

[18] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, et al. Memserve: Context caching for disaggregated llm serving with elastic memory pool. *arXiv preprint arXiv:2406.17565*, 2024.

[19] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, et al. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181*, 2024.

[20] Jordan Juravsky, Bradley Brown, Ryan Ehrlich, Daniel Y Fu, Christopher Ré, and Azalia Mirhoseini. Hydragen: High-throughput llm inference with shared prefixes. *arXiv preprint arXiv:2402.05099*, 2024.

[21] Byeongho Kim, Sanghoon Cha, Sangsoo Park, Jieun Lee, Sukhan Lee, Shin-haeng Kang, Jinin So, Kyungsoo Kim, Jin Jung, Jong-Geon Lee, et al. The breakthrough memory solutions for improved performance on llm inference. *IEEE Micro*, 2024.

[22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[23] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. {InfiniGen}: Efficient generative inference of large language models with dynamic {KV} cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 155–172, 2024.

[24] Yunjae Lee, Jinha Chung, and Minsoo Rhu. Smartsage: training large-scale graph neural networks using in-storage processing architectures. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pages 932–945, 2022.

[25] Xing Li, Zeyu Xing, Yiming Li, Linping Qu, Hui-Ling Zhen, Wulong Liu, Yiwu Yao, Sinno Jialin Pan, and Mingxuan Yuan. Kvtuner: Sensitivity-aware layer-wise mixed precision kv cache quantization for efficient and nearly lossless llm inference. *arXiv preprint arXiv:2502.04420*, 2025.

[26] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. Minicache: Kv cache compression in depth dimension for large language models. *arXiv preprint arXiv:2405.14366*, 2024.

[27] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 38–56, 2024.

[28] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Advances in Neural Information Processing Systems*, 36, 2024.

[29] Microsoft. Deepspeed-mii, 2024. GitHub repository.

[30] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5, 2023.

[31] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. vattention: Dynamic memory management for serving llms without pagedattention. *arXiv preprint arXiv:2405.04437*, 2024.

[32] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation—a {KVCache-centric} architecture for serving {LLM} chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, 2025.

[33] Keshav Santhanam, Deepti Raghavan, Muhammad Shahir Rahman, Thejas Venkatesh, Neha Kunjal, Pratiksha Thaker, Philip Levis, and Matei Zaharia. Alto: An efficient network orchestrator for compound ai systems. *arXiv preprint arXiv:2403.04311*, 2024.

[34] ShareGPT. Sharegpt. https://sharegpt.com.

[35] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E Gonzalez, and Ion Stoica. Fairness in serving large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 965–988, 2024.

[36] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: high-throughput generative inference of large language models with a single gpu. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.

[37] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 590–606, 2024.

[38] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. *arXiv preprint arXiv:2406.03243*, 2024.

[39] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, et al. Demystifying cxl memory with genuine cxl-ready systems and devices. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 105–121, 2023.

[40] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey, 2022.

[41] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[42] Haoran Wang, Haobo Xu, Ying Wang, and Yinhe Han. Cta: Hardware-software co-design for compressed token attention mechanism. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 429–441. IEEE, 2023.

[43] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 640–654, 2024.

[44] Lingfeng Xiang, Zhen Lin, Weishu Deng, Hui Lu, Jia Rao, Yifan Yuan, and Ren Wang. Matryoshka: Non-exclusive memory tiering via transactional page migration. *OSDI*, 2024.

[45] Yi Xiong, Hao Wu, Changxu Shao, Ziqing Wang, Rui Zhang, Yuhong Guo, Junping Zhao, Ke Zhang, and Zhenxuan Pan. Layerkv: Optimizing large language model serving with layer-wise kv cache management. *arXiv preprint arXiv:2410.00428*, 2024.

[46] Dongjie Yang, XiaoDong Han, Yan Gao, Yao Hu, Shilin Zhang, and Hai Zhao. Pyramidinfer: Pyramid kv cache compression for high-throughput llm inference. *arXiv preprint arXiv:2405.12532*, 2024.

[47] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 94–109, 2025.

[48] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.

[49] Zhenyu Zhang, Shiwei Liu, Runjin Chen, Bhavya Kailkhura, Beidi Chen, and Atlas Wang. Q-hitter: A better token oracle for efficient llm inference via sparse-quantized kv cache. *Proceedings of Machine Learning and Systems*, 6:381–394, 2024.

[50] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36, 2024.

[51] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems*, 37:62557–62583, 2024.

[52] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. *OSDI*, 2024.